

# **2025 Jan Oppe 1 - Set 2**

**Section 3 - problem 2**

## Pattern Printing - Z Pattern

You are tasked with drawing a Z-shaped pattern using asterisks (\*). The Z pattern is a right-angled triangle with horizontal lines at the top and bottom, and a diagonal line from the top-right to the bottom-left.

Note that there should be no space after the last '\*' in a line.

### PROBLEM STATEMENT

You are tasked with drawing a Z-shaped pattern using right-angle asterisks (\*). The Z pattern is a right-angled triangle with horizontal lines at the top and bottom, from the top-right to the bottom-left.

Note that there should be no space after the last '\*' in a line.

#### Constraints:

A single integer  $n$  representing the size of the Z pattern (i.e., the number of rows and columns). It is guaranteed that  $n \geq 3$ .

### EXAMPLE & EXPLANATION

Given the input  $n = 5$ :

#### Explanation:

- The pattern will have 5 rows and 5 columns.
- Row 1: 5 asterisks.
- Rows 2, 3, 4: A diagonal line of asterisks from top-right to bottom-left.
- Row 5: 5 asterisks.

#### Output:

```
*****
 *
 *
 *
*****
```

# THE PROBLEM SPECIFICATION

## INPUT

**n (int)**

Constraint:  $n \geq 3$

## THE BLACK BOX

1. Construct a square pattern of size  $n \times n$ .
2. Fill top & bottom rows with '\*'.
3. Fill the anti-diagonal with '\*'.
4. Ensure no trailing spaces in any row.

## OUTPUT

**List of Strings**

Each string represents one row of the Z-pattern.

---

**NOTE:** This is an I/O question. Take standard input 'n' from standard input and print the pattern row strings to standard output.

# Visualizing Z-Pattern Logic

Example Data:  $n = 3$

## Top & Bottom Rows

Row 0 (Top)

|   |   |   |
|---|---|---|
| * | * | * |
|---|---|---|

Row 2 (Bottom)

|   |   |   |
|---|---|---|
| * | * | * |
|---|---|---|

Full rows of asterisks at the start and end.

## Anti-Diagonal (Middle Row)

Row 1

|  |   |  |
|--|---|--|
|  | * |  |
|--|---|--|

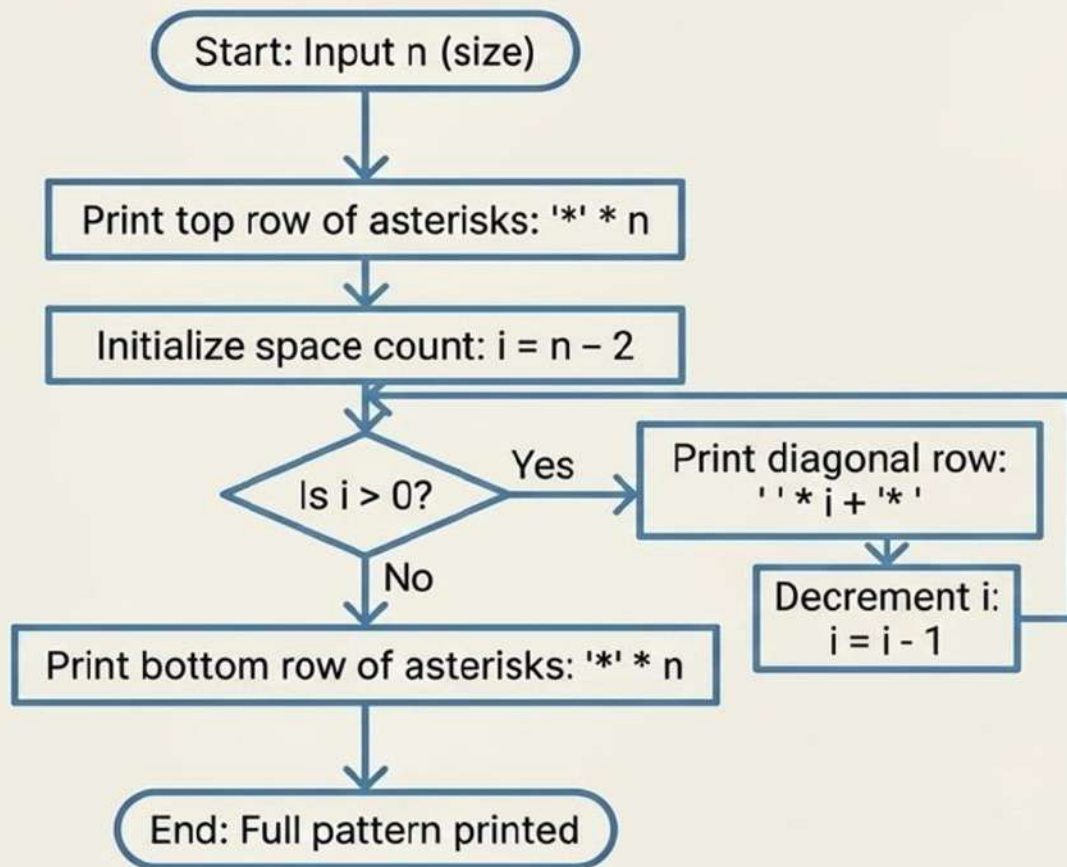
Single asterisk at  $\text{col} = n - 1 - \text{row}$  (Position 1).

## Final Combined Pattern

|   |   |   |
|---|---|---|
| * | * | * |
|   | * |   |
| * | * | * |

**Result:** 3x3 Z-Pattern

# The Logic Flowchart





# The Explanatory Approach

Step-by-step implementation for clarity.

```
# Step 1: Read pattern size 'n'  
n = int(input())
```

```
# Step 2: Print top row  
print('*' * n)
```

```
# Step 3: Loop for diagonal rows  
for i in range(n - 2, 0, -1):  
    # Step 4: Print each diagonal row  
    print(' ' * i + "*")
```

```
# Step 5: Print bottom row  
print('*' * n)
```

← Read the user's input, a single integer 'n', defining the pattern size.

← Print a full horizontal line of 'n' asterisks for the top of the Z.

← Loop through the middle rows to create the Z's diagonal.  
'i' represents *decreasing* space counts.

← Construct and print each middle row:  
'i' leading spaces followed by a single "\*".

← Finish the Z by printing a final full horizontal line of 'n' asterisks.

# The Pythonic Optimization

Leveraging built-ins for concise logic.

Type Hinting  
for clarity.

```
print('*' * n + '\n' + '\n'.join(' ' * (n - 2 - i)  
    + '*' for i in range(n - 2)) + ('\n' + '*' * n  
    if n > 1 else ''))
```

String Joining &  
List  
Comprehension.

Efficient one-liner for complete  
output and line spacing.

Result: Identical output, cleaner syntax.

# Concept Spotlight: Iteration & Boundaries

Using loops to traverse data and conditional checks to handle edge cases.

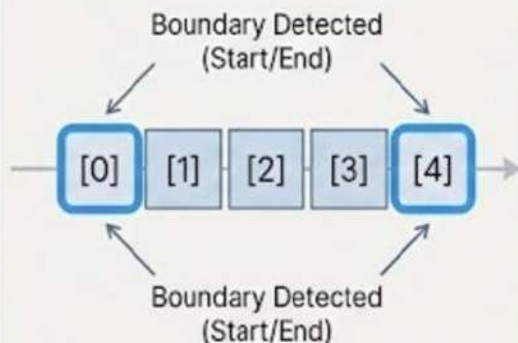
## Concept Explanation

In Python, `for` loops allow us to repeat actions, often iterating through a range or sequence. Boundary conditions are `if` statements placed within the loop to identify and treat specific points—like the first or last item—differently from the rest.

## Code View

```
for i in range(5):  
    if i == 0 or i == 4:  
        ... # Handle boundaries
```

## Runtime Object

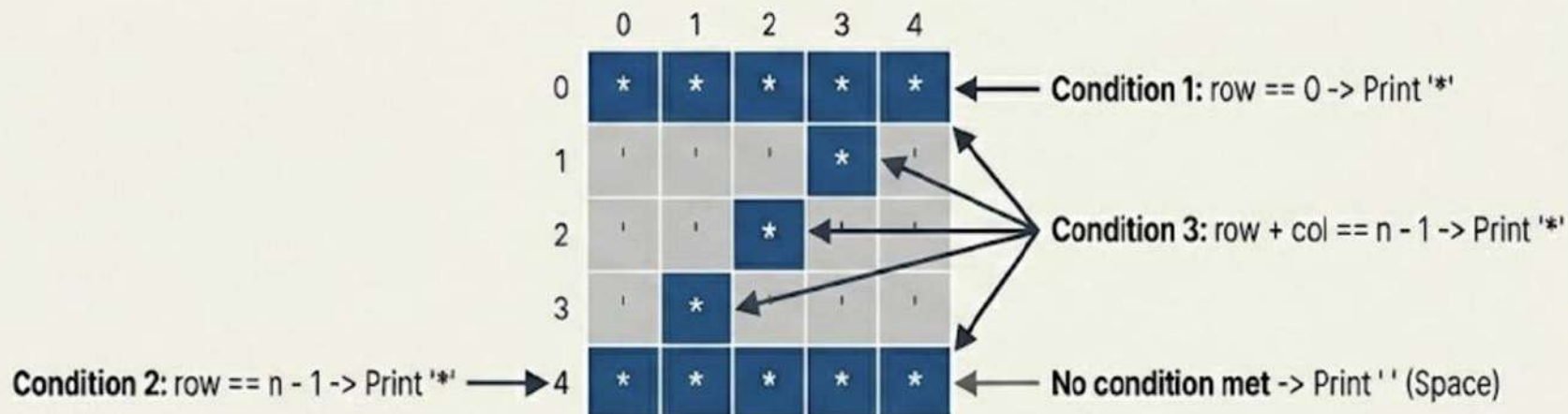


**Application:** Printing a frame. **Input:** `n=3`. **Output:** Top/Bottom rows are full (`***`), middle rows are hollow (`* *`).



# Concept Spotlight: Conditional Pattern Logic (Z-Pattern)

Using coordinates to determine the character for each grid position.



## Scenario A: Top Row ( $n=5$ )

Position: (0, 2)

Check 1:  $row == 0?$  ( $0 == 0$ )  $\rightarrow$  True

Result: Print '\*'

(No need to check anti-diagonal)

## Scenario B: Anti-Diagonal ( $n=5$ )

Position: (2, 2)

Check 1:  $row == 0$  or  $4?$  ( $2 == 0$ ,  $2 == 4$ )  $\rightarrow$  False

Check 2:  $row + col == 4?$  ( $2 + 2 == 4$ )  $\rightarrow$  True

Result: Print '\*'

## Scenario C: Empty Space ( $n=5$ )

Position: (1, 2)

Check 1:  $row == 0$  or  $4?$  ( $1 == 0$ ,  $1 == 4$ )  $\rightarrow$  False

Check 2:  $row + col == 4?$  ( $1 + 2 == 3$ )  $\rightarrow$  False

Result: Print ' ' (Space)

**Robustness Rule:** Clearly define mutually exclusive conditions (e.g., using if-elif-else) to ensure only one character is chosen for each position.

# Solution Comparison

## Step-by-Step Approach

- Easy to read and understand.
- Uses intermediate steps for clarity.

```
n = int(input())
print('*' * n)
for i in range(n - 2, 0, -1):
    print(' ' * i + '*')
print('*' * n)
```

## Concise One-Liner

- More compact solution.
- Generates the pattern using string joins.

```
print('*'*n + '\n' +
      '\n'.join(' '*(n-2-i) + '*'
                for i in range(n-2)) +
      ('\n' + '*'*n if n > 1 else ''))
```

The solution prints a full row of stars at the top and The middle rows place a single '\*' moving diagonal top-right to bottom-left to form the Z pattern.

# Key Takeaways

01

## Problem: Z-Pattern Printing

Generate a square grid pattern with a full top row, a full bottom row, and a diagonal from top-right to bottom-left using asterisks.

02

## Core Logic: Conditional Checks

Use conditional logic (e.g., `if/else` or list comprehension) to print a `\*` if the position is in the top row, bottom row, or on the anti-diagonal.

03

## Return Value: Multi-line String

The function must return a single string containing the entire pattern, with each row separated by a newline character (`\n`).

## Final Syntax Cheat Sheet

```
return "\n".join(['*' * n if i == 0 or i == n - 1 else ' ' * (n - 1 - i) + '*' + ' ' * i for i in range(n)])
```